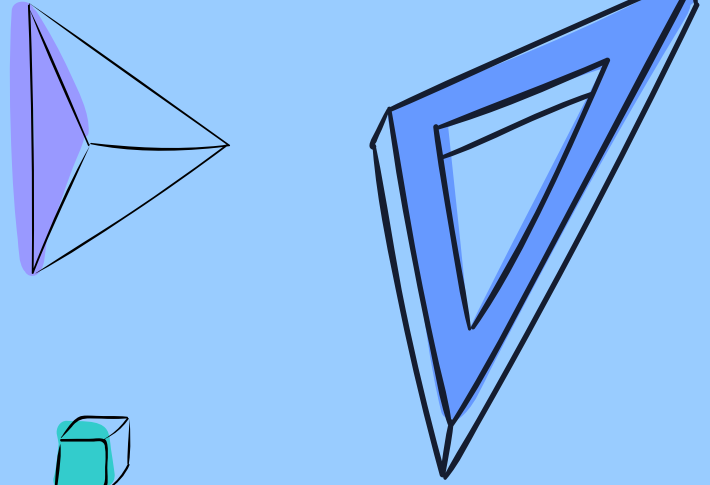
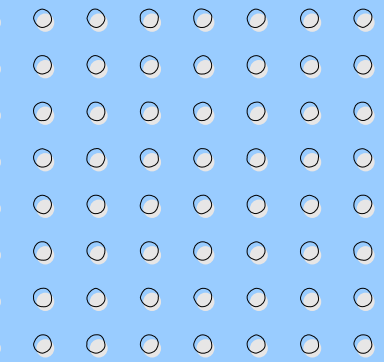
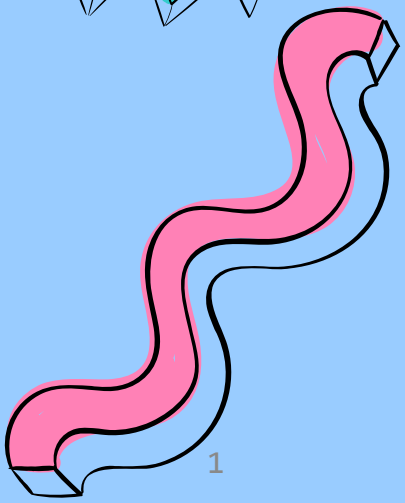


Lecture 6

Iteration Statements

PART I: Basics

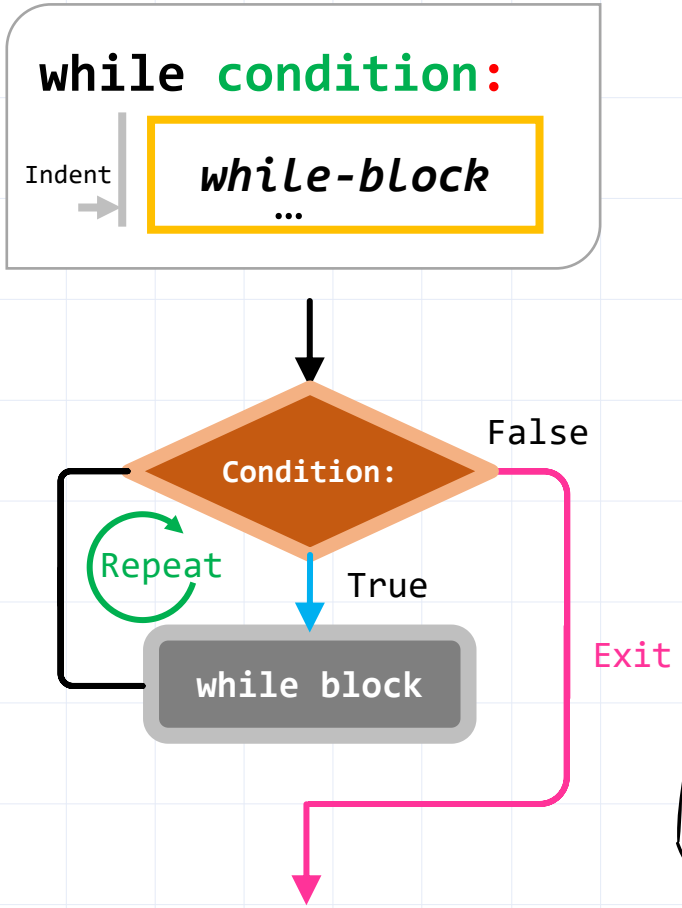





01 while loop

[1] while loop

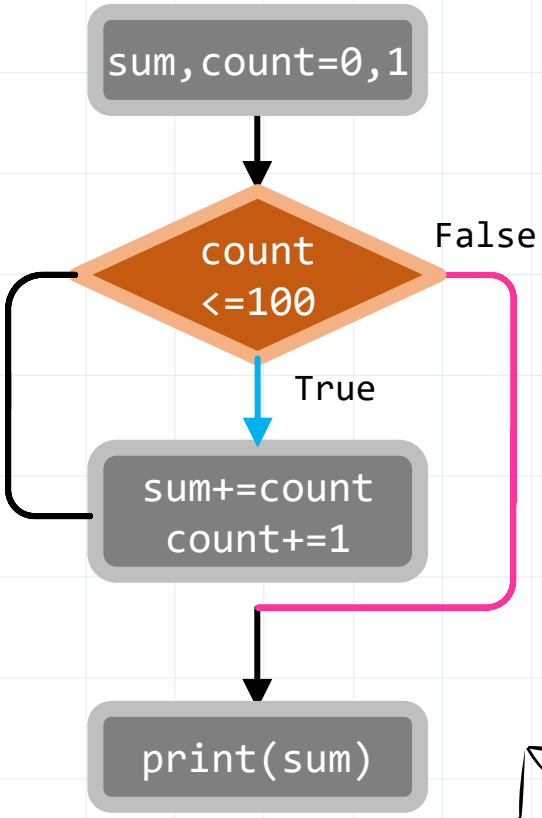
- ***while*** statement
  - Repeat a ***while-block*** while the condition is true.
    - ***while-block*** is identified by the indentation.
  - Comparison to ***if*** statement
    - ***if*** executes the block **once** if the condition is met.
    - ***while*** repeats the block if the condition is met



[2] Example

- Adding from 1 to 100
  - Increase count by 1 to be used in stop condition.

```
sum, count = 0, 1
while count <= 100:
        sum += count
        count += 1
print(sum)
```



Infinite loop

```
while True :
    while-block
    …
```

[2] Example

Save money

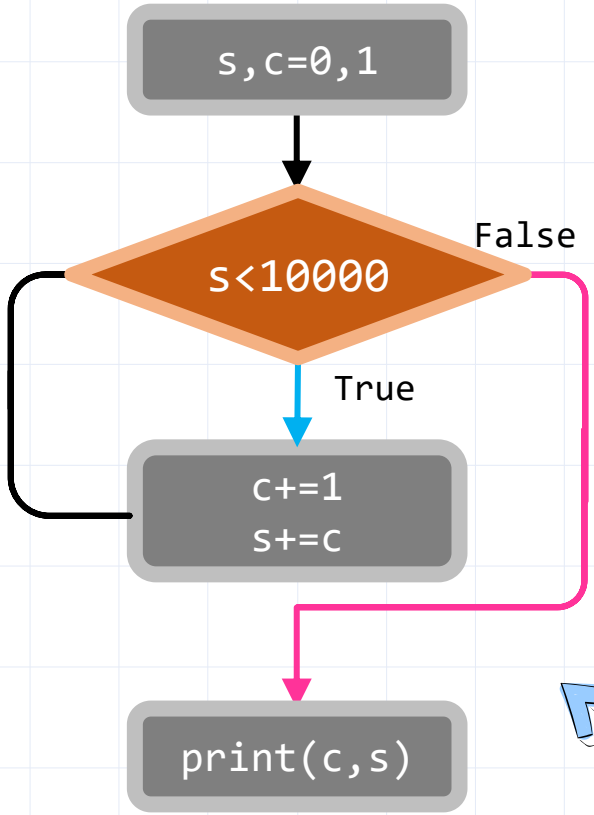
- Suppose you save $1 on the first day, $2 on the next day, and so on. If you save money like this, write a program to find out how many days later the sum exceeds $10,000.

[2] Example

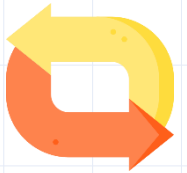
- Flowchart

```
s, c = 0, 0
while s <= 10000:
        c += 1
        s += c
print(f'On day {c}, the total amount becomes ${s}.')
```

```
On day 141, the total amount will be $10011.
>>>
```

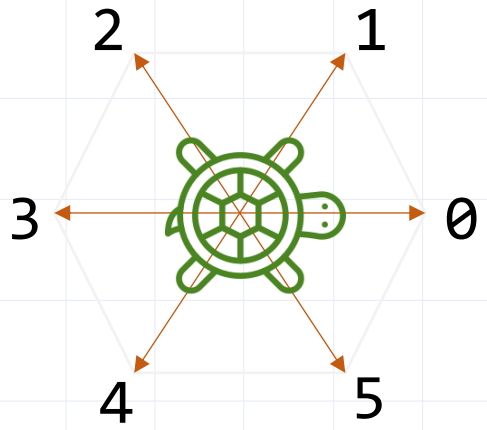


[2] Example



Wandering turtle

- Move forward the turtle by 10.
- After moving forward once, use the random module to change the direction of the turtle's progress, selecting one of 0° , 60° , 120° , 180° , 240° , or 300° .
- Repeat the above process indefinitely.



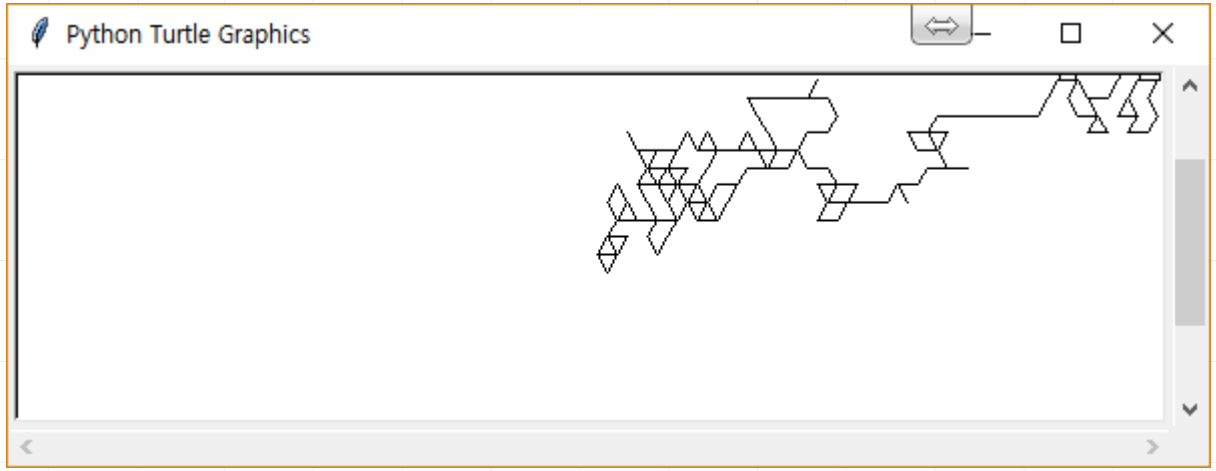
[2] Example

- Code

```
import turtle as t
import random as r
srn = t.Screen()
srn.setup(600, 200)
tu = t.Turtle()
run = True
while(run):
    tu.setheading(r.randint(0, 5)*60)
    tu.forward(10)
```

[2] Example

- Result



randint in random module

- ***randint****(start,end)* : Generate a random integer number in the range from start to end.
- *randint(0,5)*: Randomly choose a number from 0, 1, 2, 3, 4, 5.

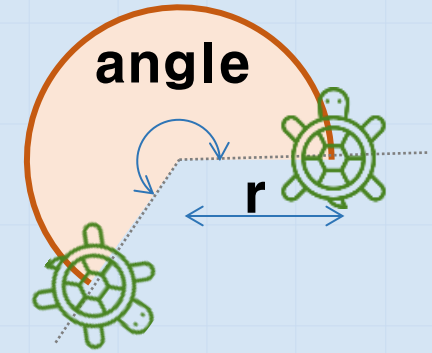
[2] Example

Create skating turtle

- First, Goes straight for 40, then, turns around a curve, smoothly following an arc with a radius of 5.
- the arc angle is limited to multiple of 60° .

circle method in turtle module

- tu.***circle***(r,angle)
- Rotate by an angle in degrees along an arc of radius r.



.circle(r,angle)

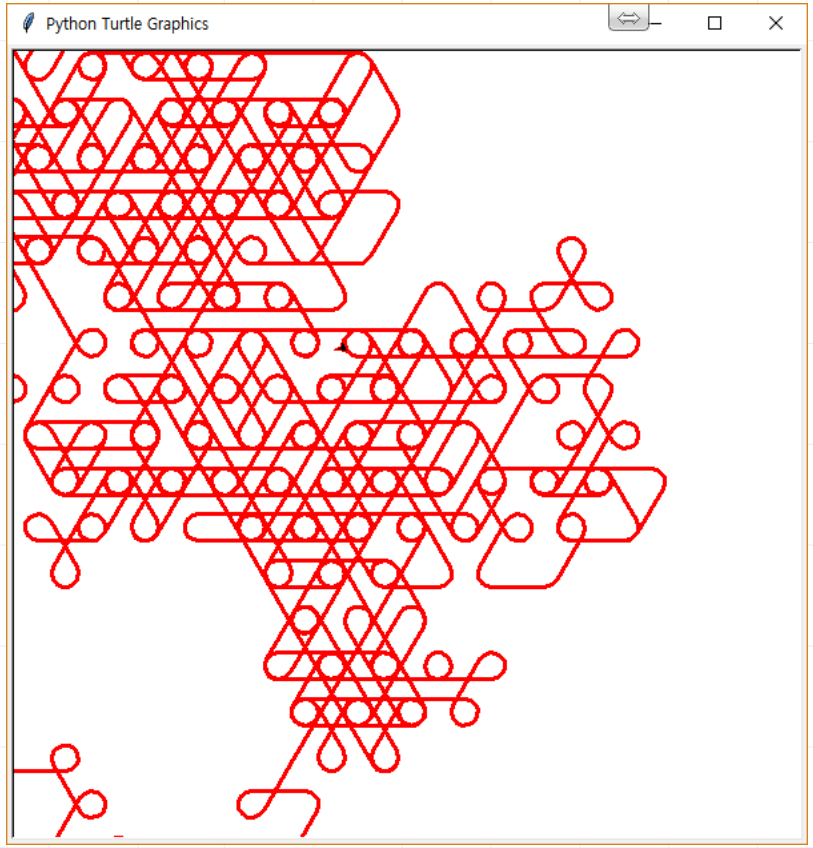
[2] Example

- Code

```
import turtle as t
import random as r
srn = t.Screen()
srn.setup(600, 600)
tu = t.Turtle()
tu.width(3)
tu.pencolor('red')
run = True
while (run):
    angle = r.randint(0, 5)*60
    tu.circle(10, angle)
    tu.forward(40)
```

[2] Example

- Result



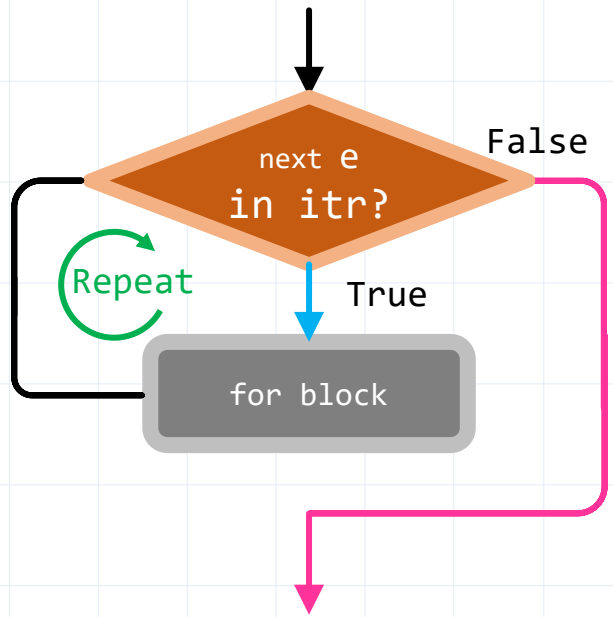
02 for loop

[1] for loop

- How to use ***for*** statement
  - Iterate through an ***iterable object*** (list, tuple, set, string, range, …)
  - Performing some action in ***for-block*** on each elements of the iterable object.
  - Repeat as many times as the number of elements in the ***iterable object***

```
for item in iterable object :
    for-block
```

Indent



[1] for loop

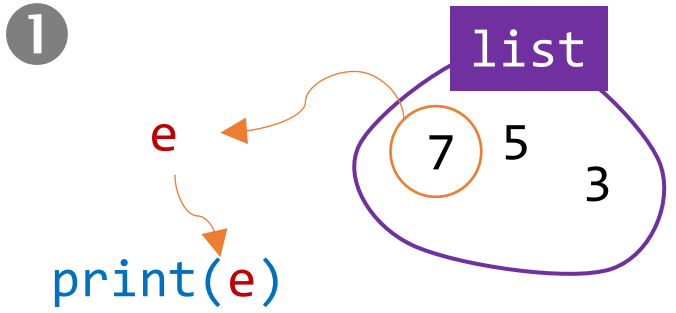
- Operation

Code

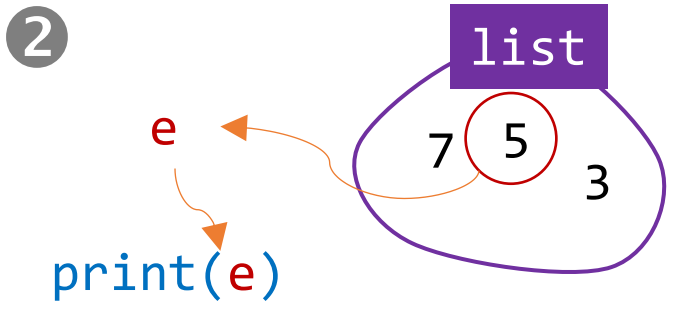
```
list = [7, 5, 3]
for e in list:
    print(e)
```

Code execution process

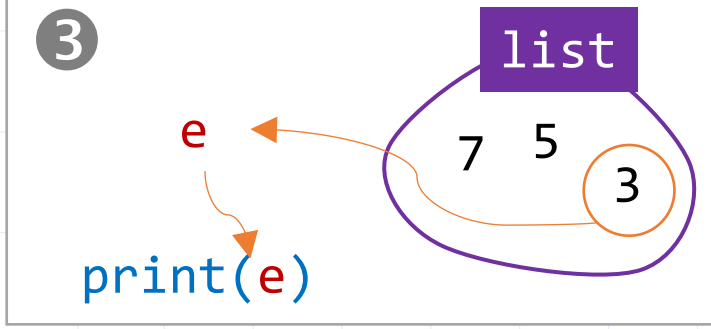
1



2



3



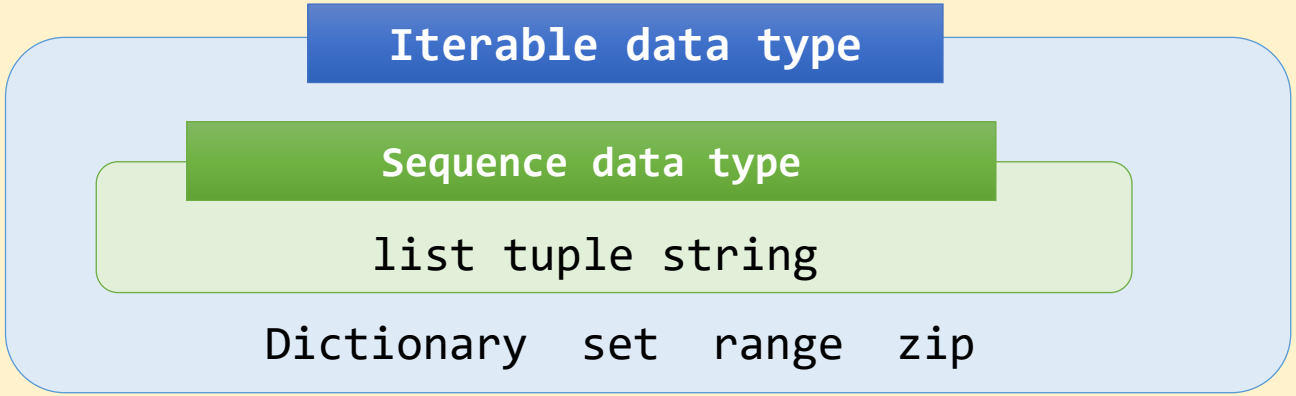
Result

```
7
5
3
```

[1] for loop

Iterable object

- All container data types including sequence data types (list, tuple, string) and non-sequence data types (dictionary, set).



- Data type is iterable if it has __iter__ attribute.
  - hasattr(str,'__iter__') → True
  - hasattr(float,'__iter__') → False

[1] for loop

Add all elements in the list using the for loop.

```
total=0
list=[1,2,8,4]
for x in list:
    total += x
print(total)
```

```
15
```

[1] for loop

Print all strings in the list.

```
# list
L=['egg','onion','carrot']
for x in L:
    print(x)
```

```
egg
onion
carrot
```

[1] for loop

Use range

```
# range
total=0
for x in range(101):
    total+=x
print(total)
```

```
5050
```

Use tuple

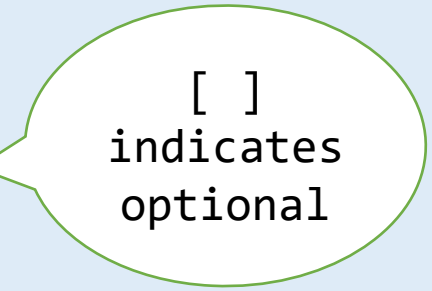
- Factorial
  - n!=1×2× ... ×n

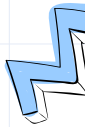
```
# tuple
import math
for i in (1,2,3,4):
    print(math.factorial(i))
```

```
1
2
6
24
```

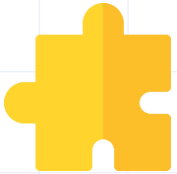
range function

- Generate a sequence of numbers
  - **range**([start=0,] stop[, step=1])
  - Usage
    - range(*stop*): Generate an integer sequence from 0 to *stop*-1
      - >>> list(range(5)) → [0, 1, 2, 3, 4]
    - range(*start*, *stop*): Generate an integer sequence from *start* to *stop* -1
      - >>> list(range(5,10)) → [5, 6, 7, 8, 9]
    - range(*start*, *stop*, *step*): Generate an integer sequence from *start* to *stop* -1 with the increment *step*
      - >>> list(range(5,10,2)) → [5, 7, 9]





[1] for loop



Use string

- Print the ASCII code for all characters in the string 'a-#@A^%'.

```
# string
for c in 'a-#@A^%':
    print(c, ord(c))
```

```
a 97
- 45
# 35
@ 64
A 65
^ 94
% 37
```

[1] for loop

ASCII code and ord() function

- **ASCII** code
  - **American Standard Code for Information Interchange**
  - 95 printable characters including spaces + 33 non-printable control characters
  - Widely used for communication between computers and devices.
- **ord**(c) function
  - Built-in function that returns ASCII code of character c
    - ex) 'A' → 65
  - chr(ascii code) : return character that corresponds to ASCII code.

ASCII code

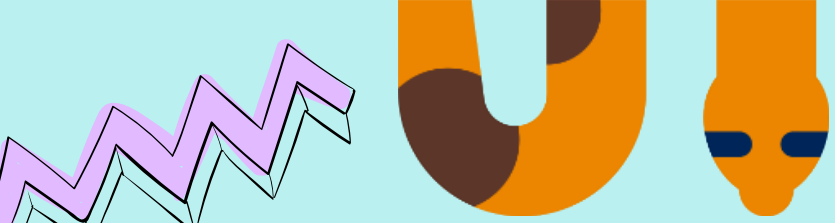
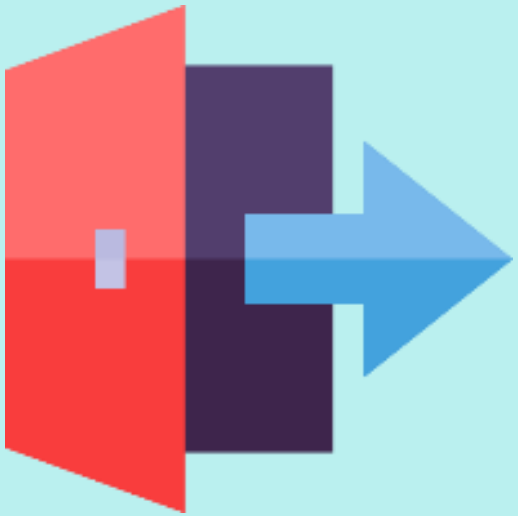
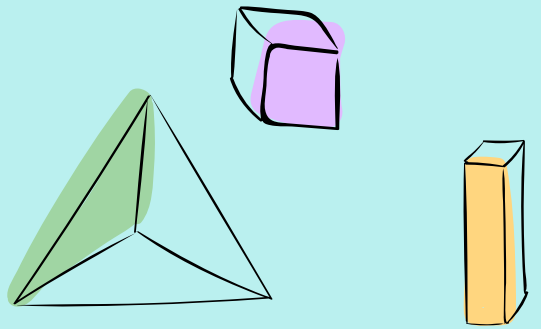
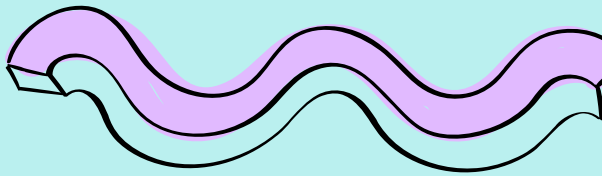
Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value	Hex	Value
00	NUL	10	DLE	20	SP	30	0	40	@	50	P	60	`	70	p
01	SOH	11	DC1	21	!	31	1	41	A	51	Q	61	a	71	q
02	STX	12	DC2	22	"	32	2	42	B	52	R	62	b	72	r
03	ETX	13	DC3	23	#	33	3	43	C	53	S	63	c	73	s
04	EOT	14	DC4	24	$	34	4	44	D	54	T	64	d	74	t
05	ENQ	15	NAK	25	%	35	5	45	E	55	U	65	e	75	u
06	ACK	16	SYN	26	&	36	6	46	F	56	V	66	f	76	v
07	BEL	17	ETB	27	'	37	7	47	G	57	W	67	g	77	w
08	BS	18	CAN	28	(	38	8	48	H	58	X	68	h	78	x
09	HT	19	EM	29	)	39	9	49	I	59	Y	69	i	79	y
0A	LF	1A	SUB	2A	*	3A	:	4A	J	5A	Z	6A	j	7A	z
0B	VT	1B	ESC	2B	+	3B	;	4B	K	5B	[	6B	k	7B	{
0C	FF	1C	FS	2C	,	3C	<	4C	L	5C	\	6C	l	7C	|
0D	CR	1D	GS	2D	-	3D	=	4D	M	5D	]	6D	m	7D	}
0E	SO	1E	RS	2E	.	3E	>	4E	N	5E	^	6E	n	7E	~
0F	SI	1F	US	2F	/	3F	?	4F	O	5F	_	6F	o	7F	DEL

[1] for loop

Print all multiples of 7 from 1 to 100 and find the sum of them.

```
sum_=0
for i in range(1,101):
    if i%7==0:
        print(i)
        sum_=sum_+i
print(f'Sum = {sum_}')
```

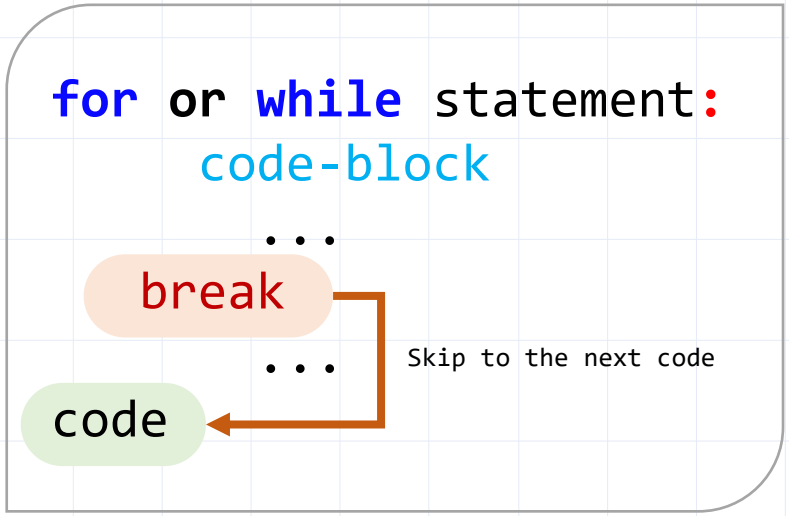
```
7
14
21
...
91
98
Sum = 735
```


03 break and continue

[1] break

- ***break*** statement
  - Loop control statement
  - Exit the loop and skip to the first code after the loop.



[1] break

Receive positive integers as input one by one. If negative integer is received, stop receiving input and print all the received numbers and their total

```
nums =[]
while True:
    num = int(input('Enter a number: '))
    if num < 0:
        break
    nums.append(num)
print(f'You entered {nums}.')
print(f'sum is {sum(nums)}.')
```

```
Enter a number: 1
Enter a number: 2
Enter a number: 4
Enter a number: 5
Enter a number: -5
You entered [1, 2, 4, 5].
The sum is 12.
```

[1] break

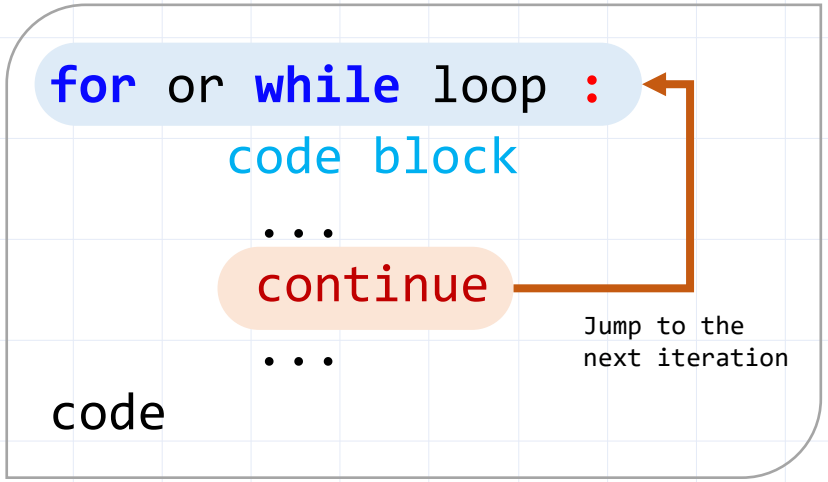
Write a program that prints all prime numbers among intergers from 2 to 10.

```
for x in range(2, 11):
    isPrime = True
    for y in range(2, int(x**0.5)+1 ):
        if x % y == 0:
            isPrime = False
            break
    if isPrime:
        print(x)
```

```
2
3
5
7
```

[2] continue

- ***continue*** statement
  - a Loop control statement
  - skips the remaining statements after the ***continue*** statement and jump to the beginning of the next iteration.

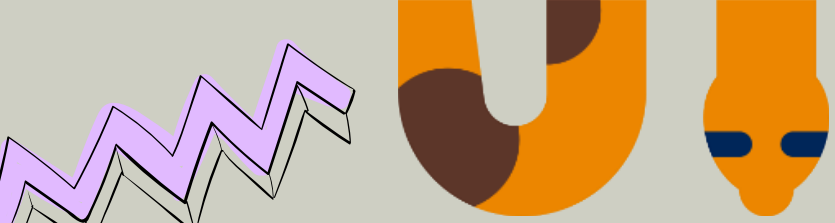


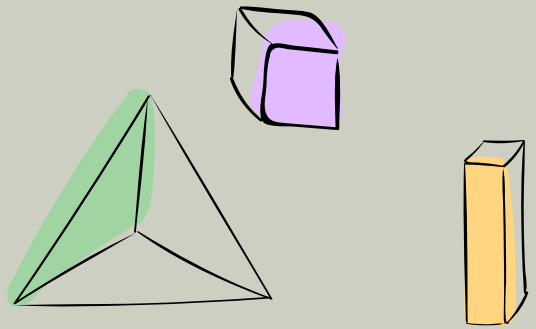
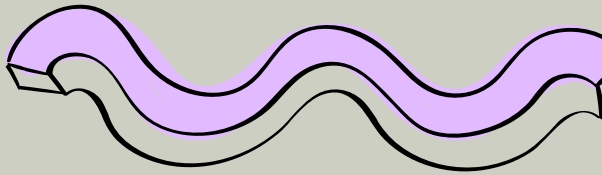
[2] continue

List all integers from 1 to 20 that are not multiples of 3 or 7.

```
nums = []
for x in range(1, 21):
    if x%3 == 0 or x%7 == 0:
        continue
    nums.append(x)
print(nums)
```

```
[1, 2, 4, 5, 8, 10, 11, 13, 16, 17, 19, 20]
```

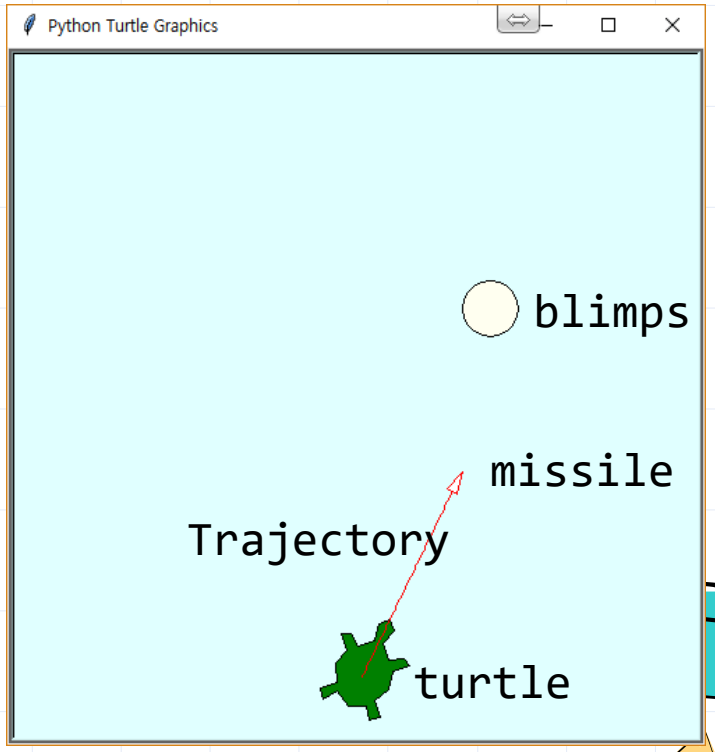


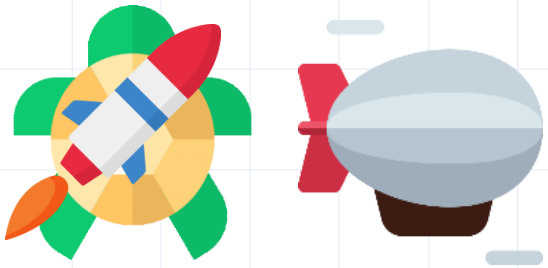

04 Missile Shooting Games

Missile Shooting

- Write a program using turtle's event functions.
  - Game Objectives
    - Change the direction of the turtle with the key and shoot down the blimp by firing missiles.
    - If the center of the missile and the center of the airship are below 30, it will be judged as a hit.
  - Turtle Object:
    - turtle(tu), blimps, missile, texter

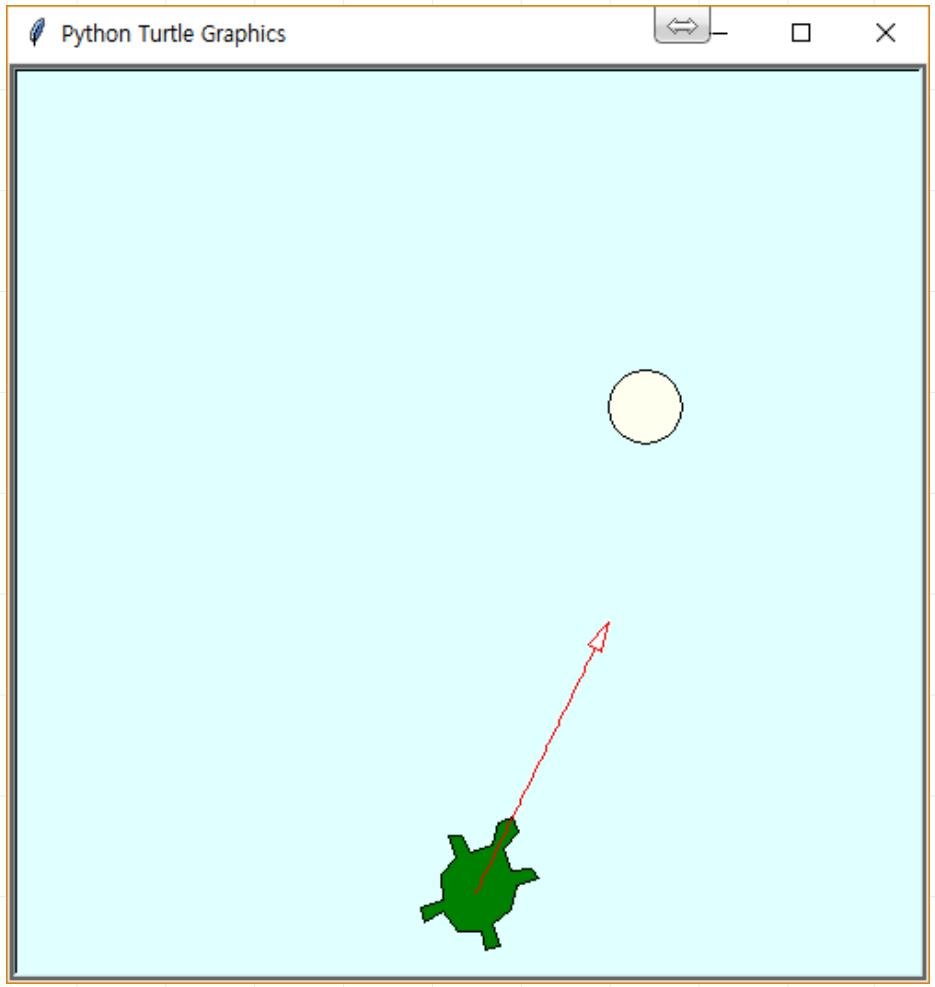


Missile Shooting

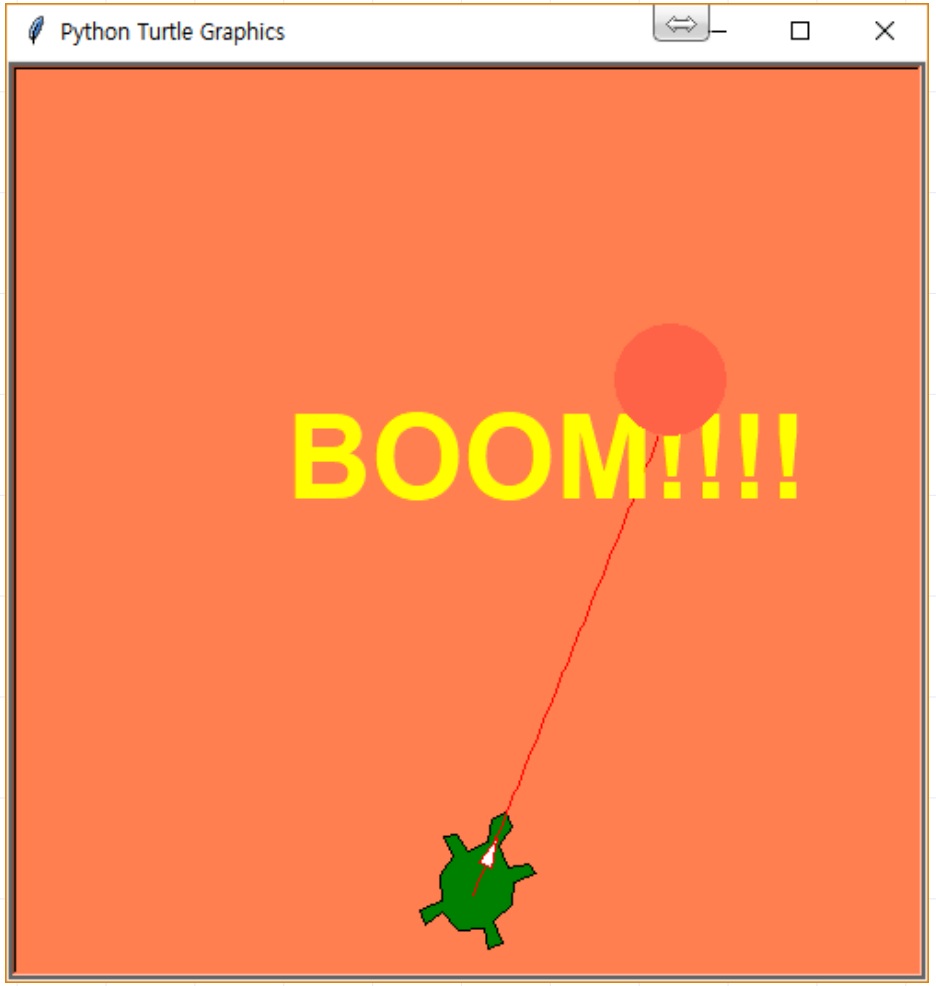


- Key Settings
  - Set the entire screen to 500x500.
  - The turtle is fixed at (0, -200) and can only change the direction of the shot.
  - The blimp has circular shape, and its initial position is in the range -200<x<200, 0<=y<=200.
- Behavior
  - Increase the angle by 2 degrees to the left with the "←" key.
  - Increase the angle by 2 degrees to the right with the "→" key.
  - Launch a missile with the "↑" key
  - Try again with the "space" key
  - Exit the program with the "ESC" key

Missile Shooting



(a) Missile launch



(a) Blimp hit

Code

[1/5]

```
import turtle as t
import random as r
SHOOT_DIST_MAX=500
def move(obj,x,y):                    #Functions to move objects
    obj.pu()                          #Pen up
    obj.goto(x,y)                     #Go to (x,y)
    obj.pd()                          #Pen down
def left():                           #Left arrow key
    global angle
    angle+=2
    tu.setheading(angle)
    missile.setheading(angle)
def right():                          #Right arrow key
    global angle
    angle-=2
    tu.setheading(angle)
    missile.setheading(angle)
```

Code

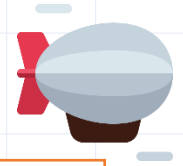
[2/5]

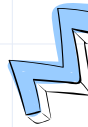
```
def shoot():                                #Up arrow key
    global trajectory
    trajectory+=3
    missile.pendown()
    missile.forward(3)
    missile.penup()
    if missile.distance(blimps)<30:         #Hit
        blimps.turtlesize(3)
        srn.bgcolor('coral')
        blimps.color('tomato','tomato')
        texter.write('BOOM!!!!',font=("Arial",48,'bold'))
        missile.goto(0,-200)
        trajectory=0
        srn.ontimer(newTrial,800)
    else:
        if trajectory>SHOOT_DIST_MAX:       #Missile finished
            missile.goto(0,-200)
            trajectory=0
            srn.ontimer(newTrial,500)
        else:                               #Missile tracking
            srn.ontimer(shoot,20)
```

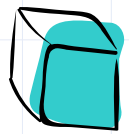
Code

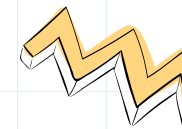
[3/5]

```
def newTrial():                              #Restart
    texter.clear()
    missile.clear()
    initBlimps()
def initBlimps():                            #Initialize Blimp
    srn.bgcolor('lightcyan')
    blimps.shape( 'circle'); blimps.turtlesize(2)
    blimps.color('black','ivory')
    move(blimps, r.randint(-200,200), r.randint(0,200))
    blimps.penup()
    direction=r.randint(0,1)
def moveBlimps():                            #Move Blimp
    global direction
    if direction:
        blimps.forward(2)
    else:
        blimps.backward(2)
    if -280 < blimps.xcor() <280:    #If the blimp is in the screen
        srn.ontimer(moveBlimps,200)
    else:                                    #If the blimp leave the screen
        initBlimps()
        srn.ontimer(moveBlimps,200)
```









Code

[4/5]

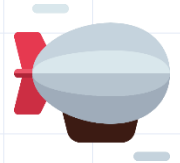
```
angle=90
direction=1
trajectory=0
#Setting Screen Objects
srn=t.Screen()
srn.setup(500,500)
srn.register_shape('missile', ((2,-3), (0,5), (-2,-3)))
srn.bgcolor('lightcyan')
# Setting the turtle object
tu=t.Turtle()
tu.setheading(angle)
tu.shape('turtle')
tu.turtlesize(3)
tu.color('black','green')
move(tu,0,-200)
# Setting the texter object
texter=t.Turtle()
texter.hideturtle()
texter.color('yellow','red')
move(texter,-100,0)
```

Code

[5/5]

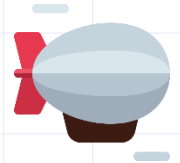
```
# Setting the missile object
missile = t.Turtle()
missile.shape('missile')
missile.color('red','white')
missile.turtlesize(2)
missile.setheading(90)
move(missile,0,-200)
#Setting the blimps object
blimps = t.Turtle()
blimps.speed(0)
initBlimps()
moveBlimps()
#screen event binding
srn.onkey(shoot, "Up")
srn.onkey(left, "Left")
srn.onkey(right, "Right")
srn.onkey(newTrial, "space")
srn.onkey(srn.bye,'Escape')
srn.listen()          # Start event service
srn.mainloop()        # Event service loop
```

Turtle's Methods for Events

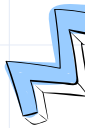


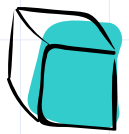
- Screen Class
  - Methods for Event Management
    - Screen_object.***listen()***
      - Start managing the event.
    - Screen_object.***mainloop()***
      - The event is processed until the program ends.

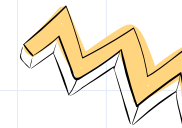
Turtle's Methods for Events



- Event Binding Methods
  - Screen_object.***onkey***(*f, key*)
    - *f*: Function that is executed when an event occurs
    - *key*: The name of the key (e.g.: 'a', '1', 'space', 'Up', 'Left', 'escape' ...)
    - Pressing key allows the function f to be executed.
  - Screen_object.***ontimer***(*f, ms*)
    - *f*: Function that is executed when an event occurs.
    - *ms*: Time in milliseconds.
    - When *ms* elapses, function *f* is executed.







Summary

- ***while***:
  - Repeat a code block while the condition is true.
  - while condition: {while block}
- ***for***:
  - Iterate through an iterable object.
  - for item in iterable object: {for block}
- ***break***:
  - Exit a loop and skip to the next code after the loop.
- ***continue***:
  - skips the remaining statements and jump to the beginning of the next iteration.